ScrollGuard 에이전트 벤치마크

Phase 2 벤치마크 설계 보고서 · Special Topics on Generative AI (Agentic AI) · 2026-11-01 (v3.1)
저장소: github.com/Blue-dogy/Advance_Topics_Generative-AI-ScrollGuard

과제 요구사항에 대한 답 요약

1. **에이전트가 할 일:** 스마트폰 사용 상황 하나를 보고 `no_intervention` / `gentle_prompt` / `ask_context` 중 하나를 고르고 근거를 JSON으로 낸다. 트랙 B에서는 터치 기록과 사용자 맥락을 **도구로 직접 조회**한 뒤 결정한다.
2. **평가 데이터:** 사례 81개(정답 32 / 31 / 18). 실제 사용 데이터(micro_datasetA, 5,777행)의 사용량·기분·평소 사용량에 사용자 상태와 5칸 관찰 기록을 결합했다. 반례(adversarial), 경계(boundary), 제스처 반례 쌍(counterfactual), 지름길 반례(shortcut)를 넣어 **숫자 하나로는 맞힐 수 없게** 설계했다.
3. **성공 기준과 지표:** 정확도(95% 신뢰구간), macro-F1, 탐지율, **불필요 개입률**, 맥락 확인율, 비용 점수, 어려운 사례 정확도, 반례 쌍 지표, 반복 실행 일관성, 도구 사용 지표. 기준선 6개와 비교해 에이전트가 "맥락을 읽어서" 더 잘하는지를 판정한다.
4. **평가 가능성:** 입력·정답 분리, 자동 채점 스크립트, 동일한 프롬프트·하네스, 누출 점검, 라벨러 간 일치도(kappa)까지 전 과정을 명령어 한 줄씩으로 재현할 수 있다.

1. 에이전트가 수행할 과제 (Task definition)

프로젝트 목표. ScrollGuard는 사용 시간만 보고 앱을 막는 잠금 앱과 달리, **사용 맥락을 읽어 의도적 사용은 방해하지 않고 무의식적 스크롤(자동조종)일 때만 부드럽게 개입**하는 에이전트다. 따라서 벤치마크는 "개입 여부를 맥락에 맞게 판단하는 능력"을 잰다.

1.1 입력과 출력

구분	내용
입력 (15열)	**상황:** 현재 시각, 진입 계기(self_open / notification / search), 앱 유형 **사용자:** 선언한 목표(입력하지 않았으면 그 이유), 사용자 상태(누구인지·어떤 날인지·지금 어디서 무엇을 하는지), 기분 자기보고(1~6), 평소 1시간 사용량 **관찰 기록:** 진입 / 앱 안 행동 / 앱 전환 / 상호작용 횟수(검색·저장·좋아요·댓글) / 일정·설정의 5칸 **사용량·제스처:** 최근 60분 사용 시간, 세션 수, 중앙 세션 길이, 사용 집중도, 스와이프 간격 변동계수(CV), 콘텐츠당 평균 체류
출력	JSON 한 줄 `{"action": "...", "reason": "..."}`. 세 행동 외의 출력은 무효(invalid)로 채점

1.2 행동 정의와 라벨 기준

행동 (의도)	기준
`no_intervention` (의도적 사용)	선언 목표와 행동이 일치하거나 목적 행동(검색·저장·작성·앱 전환·결제 등)이 관찰됨. 짧은 유틸리티 사용, 스스로 정한 시간 안의 사용. **오래 썼다는 것만으로는 개입하지 않음.**
`gentle_prompt` (자동조종)	독립된 자동조종 신호가 **2개 이상**이고 반대 증거가 없음. 신호 예: 목적 행동 부재, 알림 후 피드로 이탈, 선언 목표·타이머·취침 목표 초과, 일정 충돌, 반복 재확인.
`ask_context` (판단 보류)	목적 신호와 이탈 신호가 충돌하거나, 사용량이 눈에 띄는데 핵심 정보에 접근할 수 없음. 잘못 개입하는 것보다 묻는 것이 낫다.

경계 규칙: 사용자의 명시적 선언은 존중한다. 시간 초과 하나만으로는 근거가 부족하다. 1~3분짜리 짧은 사용은 정보가 없어도 질문하지 않는다.

1.3 두 개의 평가 트랙

트랙	에이전트가 하는 일	실행 스크립트
A. 단일 결정	15개 입력을 모두 받고 한 번에 결정한다.	`run_agent.py`
B. 터치 트랙 (도구 사용 에이전트)	처음에는 시각·진입 계기·앱 유형·사용량 숫자만 본다. 나머지는 도구를 호출해 스스로 모은다(사례당 최대 4회). `get_touch_log` 최근 스와이프의 간격(초)·속도(px/s)·방향 · `compute_stats` 평균·CV 등 계산 · `get_user_context` 선언 목표, 사용자 상태, 기분, 관찰 기록 "무엇을 확인한 뒤 결정했는가"라는 **궤적(trajectory)**까지 기록·채점한다.	`run_agent_tools.py` `build_touch_logs.py`

1.4 감지 층에 대한 가정

실제 앱은 **감지 층**(사용 기록·알림·접근성 이벤트·캘린더 권한으로 로그를 모으는 부분)과 **판단 층**(모은 정보로 행동을 정하는 에이전트)으로 나뉜다. 이 벤치마크는 **판단 층**을 평가한다. 관찰 기록 열은 감지 층이 이미 정확히 동작해 만든 출력이라고 가정하며, 감지 정확도·개인정보 부담·iOS 제약은 평가 범위 밖이다.

입력	실제 수집 방법 (안드로이드 기준)
앱, 사용 시간, 세션 수, 앱 전환	UsageStatsManager (사용 기록 권한)
진입 계기	NotificationListenerService로 알림 직후 실행인지 확인
스와이프 간격, 체류, 검색·저장·좋아요 등	AccessibilityService의 스크롤·클릭 이벤트와 화면 이름
일정, 취침 목표, 타이머, 선언 목표	캘린더 권한, ScrollGuard 앱 안 설정과 목표 질문

1.5 피치 덱(Phase 1)과의 대응

피치 덱의 내용	벤치마크에서의 반영
Context-Blind (기존 잠금 앱은 연구와 무의식적 스와이프를 구분하지 못함)	과제 자체가 이 구분이다. 시간 기준 규칙(B1)을 기준선으로 두어 불필요 개입률 41%로 한계를 수치화하고, adversarial 사례로 시간만 보면 틀리게 만들었다.
Gesture Tracking (스크롤 속도·멈춤·방향)	스와이프 간격 CV와 평균 체류(트랙 A), 간격·속도·방향 시퀀스(트랙 B의 `get_touch_log`). 멈춤은 간격 시퀀스의 긴 간격으로 드러난다.
Zombie Detection (균일하고 기계적인 스와이프)	균일한 스와이프를 핵심 신호로 쓰되, 덱의 주장을 한 단계 다듬었다. 웹툰 정주행·사진 정리처럼 **균일하지만 의도적인** 경우가 있어 제스처 반례 쌍 12개로 "제스처만 보면 틀린다"를 검증한다 (B3: pair_both_correct 0.00).
Contextual Analysis (언제·왜·어떻게 앱을 쓰는가)	선언 목표, 사용자 상태, 5칸 관찰 기록, 평소 사용량(실제 값), 일정·취침 목표
Silent Operation (고정된 시간 잠금 없음)	행동 공간에 차단이 없다. 기본값은 `no_intervention`이고 불필요 개입에 가장 큰 비용(1.0)을 준다.
Grayscale Fade · Gentle Friction · Restored Choice (팝업 또는 TTS)	`gentle_prompt` = 흑백 전환과 작은 팝업·TTS 확인, 사용자는 언제든 계속 가능(에이전트 프롬프트에 명시). `ask_context` = 의도를 먼저 묻는 가벼운 마찰.
"강제가 아니라 스스로 멈추게 돕는다"	불필요 개입률(false_intervention)을 핵심 지표로 두고, 질문도 방해로 보아 비용을 매긴다(3.2절).

원시 터치 좌표를 분류하는 문제는 시계열 모델 평가이므로 과제(에이전트 평가)에 맞지 않는다. 대신 터치 기록을 **에이전트가 필요할 때 조회하는 관찰 도구**로 바꿔, 강의에서 말한 에이전트 벤치마크의 형태(도구·여러 턴·궤적)로 만들었다.

2. 평가 데이터 (Test cases)

2.1 구성

사례 유형	설계 의도	no_int.	gentle	ask	합계
standard	전형적인 사례	10	13	5	28
adversarial	사용 시간·세션 수·집중도만 보면 틀리게 설계 (예: 50분 시청이지만 시험 공부)	6	3	0	9
boundary	신호가 섞이거나 정보가 막힌 경계 사례	2	1	13	16
counterfactual	같은 맥락·같은 사용량에 **제스처만 반대로 바꾼 쌍** 6개. 정답은 맥락으로 정해지므로 쌍의 정답이 같다	6	6	0	12
shortcut	체류 시간·CV·목표 선언 유무·앱 유형·제스처 결측·심야·문장 길이 같은 **단일 단서 지름길과 반대 방향**인 사례	8	8	0	16
합계		**32**	**31**	**18**	**81**

2.2 데이터 출처와 생성 방법

- **실제 데이터:** 사용 시간·세션 수·중앙 세션 길이·사용 집중도 4개 숫자는 공개 데이터 `micro_datasetA.csv` (5,777행 × 17열)의 실제 행에서 시드 고정(42)으로 추출했다. 같은 행의 **기분 자기보고(valence)**와, 같은 사용자의 다른 측정 기록(평균 약 18회)에서 구한 **평소 1시간 사용량 중앙값**도 실제 값이다. 평소보다 많이 쓴 의도적 사용과 평소만큼 쓴 자동조종이 모두 있어, 사용량 비교만으로는 판단할 수 없다.
- **합성 데이터:** 시각, 진입 계기, 앱 유형, 스와이프·체류 값, 선언 목표, 사용자 상태(`user_states.py`), 관찰 기록은 라벨 기준에 맞춰 직접 작성했다. 관찰 기록은 **관찰된 사건과 횟수만** 적는다("검색 없음" 같은 요약 표현을 쓰지 않음). 예: *진입: 추천 영상 알림 탭 / 앱 안: 추천 피드에서 쇼츠 412개 연속 재생 / 앱 전환: — / 상호작용: 검색 0, 저장 0, 좋아요 0, 댓글 0 / 일정·설정: 내일 09:00 수업.* 목표를 입력하지 않은 49개 사례는 빈칸 대신 그 이유(질문 건너뜀, 기능 꺼 둠, 창 닫음)를 정답과 무관하게 배정했다. 트랙 B의 터치 기록(사례당 스와이프 8~40개)은 간격·속도·방향으로 이루어진다. 간격은 각 사례의 CV·평균 체류와 ±10% 안으로 맞춰 로그정규분포에서 생성하고, 속도는 간격이 일정할수록 일정하게, 방향(뒤로 가기 비율 0~15%)은 정답과 무관하게 무작위로 정했다.
- **언어:** 공식 평가 언어는 영어(`*_en.csv`)다. 한국어 원문에서 번역했고, 숫자·범주 열이 한국어판과 같은지와 문장 속 숫자가 일치하는지를 스크립트로 검증한다.

2.3 데이터 품질 관리

- **입력과 정답 분리:** 에이전트는 `benchmark_inputs_en.csv` 만 받는다. 정답(`benchmark_labels.csv`)과 라벨 근거·사례 유형(`benchmark_meta*.csv`)은 채점에만 쓴다.
- **정답 누출 점검:** `check_leakage*.py` 가 입력 문장에서 정답을 드러내는 표현("intentional", "autopilot", "addict", "의도적", "중독" 등)을 찾는다. 현재 0건이다.
- **정합성 점검:** `check_consistency.py` 가 관찰 기록과 숫자 열이 모순되지 않는지 확인한다(진입 계기, 반복 실행 횟수 = 세션 수, 앱 안 시간 ≤ 사용 시간, 항목 수 × 평균 체류 ≤ 사용 시간). 81개 모두 통과한다. 이 점검으로 "52분 영상을 끝까지 봄"인데 사용 시간이 47.7분인 사례를 찾아 고쳤다.
- **라벨 규칙 점검:** 81개 사례를 라벨 기준에 대조했다. gentle_prompt 3개(S002, S022, S044)가 독립 신호 1개뿐이어서 일정 충돌·취침 목표 초과를 관찰 기록에 추가해 규칙(신호 2개 이상)을 만족시켰다.
- **지름길 점검:** 맥락을 이해하지 않고 숫자·범주·단어 하나로 맞히는 규칙의 최고 정확도를 측정했다. 초기 45개에서는 CV 임계값 하나로 0.76이 나와 결함을 발견했고, 반례 쌍과 지름길 반례를 추가했다. 또 이전 맥락 문장은 "no search, no saves"처럼 기준을 요약해 적어 **부정어가 있으면 개입**이라는 규칙만으로 0.60이 나왔다. 그래서 관찰 기록 형식으로 다시 썼다.

단일 단서 규칙	이전 버전*	현재 (v3.1)
스와이프 CV 임계값 (`CV < 0.6` 이면 개입)	**0.76**	0.49
평균 체류 임계값 (가장 좋은 임계값)	**0.72**	0.53
선언 목표 유무, 앱 유형, 심야 여부, 문장 길이 등	0.6 안팎	0.43~0.54
문장에 부정어(no, not, without)가 있으면 개입	**0.60**	0.40
기분, 평소 사용량, 평소 대비 사용 비율 (새 열)	-	0.44~0.49
B4 단어 분류기 (정답으로 학습, 진단용)	**0.72**	0.60
다수 클래스 기준선 (항상 가장 많은 정답)	0.42	0.40

* CV·체류·목표 유무 등은 초기 45개, 부정어 규칙과 B4는 관찰 기록으로 바꾸기 전 81개 버전에서 잰 값이다.

- **라벨 독립 검증:** 설계에 참여하지 않은 2명이 `annotation_sheet_en.csv` 를 정답 없이 각자 채우고, `agreement.py` 로 Cohen's kappa를 계산한다. 0.6 이상이면 라벨이 타당하다고 보고, 0.4 미만이면 라벨 기준을 다시 정한다.

3. 성공 기준과 지표 (Criteria & metrics)

3.1 지표

지표	정의	왜 필요한가
accuracy (+95% CI)	전체 정답률, 부트스트랩 2,000회로 신뢰구간	기본 성능. 81개라 구간이 넓어 차이 해석에 필요
macro-F1, 혼동행렬	세 행동별 F1의 평균	한 행동만 찍는 편향을 드러냄
detect_rate	gentle_prompt 사례를 맞힌 비율	자동조종을 놓치지 않는가
false_intervention	no_intervention 사례에 gentle_prompt를 낸 비율	**제품 철학의 핵심.** 의도적 사용 방해는 신뢰 손실
question_burden	no_intervention 사례에 ask_context를 낸 비율	불필요한 질문도 사용자 부담
ask_recall	ask_context 사례를 맞힌 비율	모호할 때 묻는 능력. 규칙 기반 기준선은 0
cost_score	1 − 평균 오류 비용 (아래 3.2절 비용표)	오류마다 피해가 다르다는 점 반영
hard_acc	adversarial·boundary·counterfactual 사례 정확도	쉬운 사례로 점수가 부풀려지는 것 방지
pair_both_correct, gesture_flip	반례 쌍 두 개를 모두 맞힌 비율 / 제스처만 바꿨는데 판단이 바뀐 쌍 비율	반사실(counterfactual) 평가. 제스처가 아니라 맥락으로 판단하는가
invalid_rate	형식 오류·무효 행동 비율 (오답으로 채점)	출력 형식 준수도 성능의 일부
consistency	같은 사례를 3회(temperature 0.3) 돌렸을 때 같은 답을 낸 비율	신뢰성. 한 번 맞힌 것과 매번 맞히는 것은 다름
도구 지표 (트랙 B)	context_check_rate, touch_check_rate, mean_tool_calls, 맥락 확인 여부별 정확도	올바른 근거를 확인하고 판단했는가 (궤적 평가)

3.2 오류 비용표 (cost_score)

정답 \ 예측	no_intervention	gentle_prompt	ask_context
no_intervention	0	1.0 (불필요 개입)	0.8 (불필요 질문)
gentle_prompt	0.8 (놓침)	0	0.5 (덜 직접적인 대응)
ask_context	0.6	0.6	0

형식 오류·무효 출력은 1.0이다. **질문(ask_context)도 사용자를 멈춰 세우는 방해**로 본다. 그래서 의도적으로 사용하는 사람에게 묻는 비용(0.8)을 개입(1.0)에 가깝게 잡았다. 처음 가중치(불필요 질문 0.5, 놓친 자동조종에 질문 0.4)에서는 모든 사례에 질문만 하는 규칙이 비용 점수 1위(0.65)가 되는 허점이 있어 수정했다. 수정 후 이 규칙은 0.49로 6개 기준선 중 5위다. 가중치는 설계 가정이므로 `evaluate.py --sensitivity`로 가중치 세트 4가지(현재, 이전, 놓침을 더 무겁게, 모든 오답 동일)에서 순위가 유지되는지 함께 보고한다. 현재 기준선 중 B2는 네 세트 모두에서 1~2위다.

3.3 성공 기준

에이전트는 다음을 **함께** 만족할 때 프로젝트 목표를 달성했다고 본다. 지표 하나만 목표로 삼으면 그 지표만 좋아지는 문제(Goodhart의 법칙)를 피하기 위해서다.

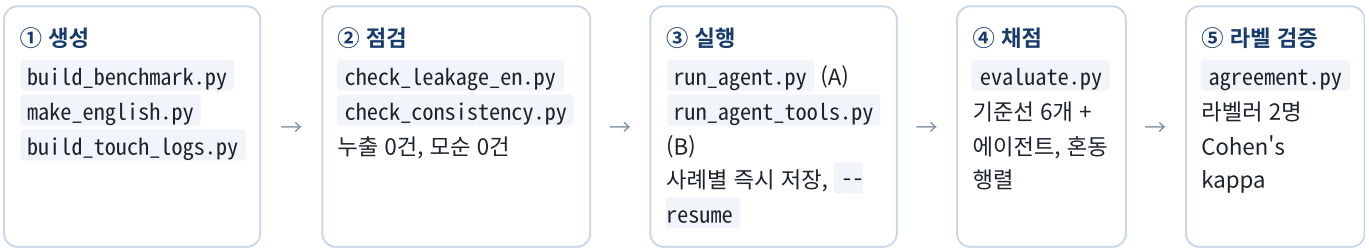
1. 정확도와 cost_score가 가장 좋은 기준선(B2: 0.51 / 0.64)보다 높을 것
2. false_intervention이 낮을 것 (시간 임계값 B1의 0.41보다 확실히 낮게, 목표 0.10 이하)
3. ask_recall이 0보다 충분히 높을 것 (규칙 기반 기준선은 모두 0)
4. pair_both_correct가 B3(0.00)보다 높고 gesture_flip이 낮을 것 (B3는 1.00)
5. invalid_rate가 낮고 consistency가 높을 것
6. 트랙 B: 맥락 확인율이 높고, 맥락을 확인한 경우의 정확도가 확인하지 않은 경우보다 높을 것

3.4 기준선 결과 (영어 입력, 81개)

기준선	accuracy	95% CI	detect	false_int.	ask_recall	hard_acc	pair_both	flip	cost
B0 항상 no_intervention	0.40	[0.30, 0.49]	0.00	0.00	0.00	0.38	0.50	0.00	0.56
B0 항상 gentle_prompt	0.38	[0.28, 0.49]	1.00	1.00	0.00	0.27	0.50	0.00	0.47
B0 항상 ask_context	0.22	[0.14, 0.32]	0.00	0.00	1.00	0.35	0.00	0.00	0.49
B1 30분 이상이면 개입	0.41	[0.30, 0.52]	0.45	0.41	0.00	0.22	0.33	0.00	0.54
B2 텍스트 없는 특징 규칙	**0.51**	[0.41, 0.60]	0.35	0.06	0.00	0.57	0.50	0.50	**0.64**
B3 스와이프 CV < 0.6	0.49	[0.40, 0.60]	0.65	0.38	0.00	0.46	0.00	1.00	0.61
B4 단어 분류기 (진단용)	0.60	[0.51, 0.70]	1.00	0.44	0.17	0.65	1.00	0.00	0.69

해석. 사용 시간 규칙(B1)은 의도적 사용의 41%에 불필요하게 개입한다. 시간만 보는 방식의 한계(맥락 무시)를 수치로 보여 준다. 어떤 규칙도 0.51을 넘지 못하므로, 에이전트가 이보다 확실히 높다면 맥락을 읽어서 얻은 성능이라고 말할 수 있다. B4는 **정답으로 학습한** 단어 분류기(사례 하나씩 빼고 학습)라 경쟁 상대가 아니라 "문장 단어에 정답 정보가 얼마나 담겼는가"의 참고치다. 남은 구분 단어는 feed, scrolled in a row(수동 소비), searched, notes(목적 행동)처럼 의미를 담은 단어이고, 부정어·문장 길이·템플릿 표현 같은 표면 단서는 제거했다. 실제 LLM 에이전트(EXAONE 3.5 7.8B, Claude) 결과는 같은 표에 행으로 추가된다.

4. 벤치마크로 에이전트를 평가하는 방법




```
python run_agent.py --lang en --inputs benchmark_inputs_en.csv --model <model> --runs 3 --out predictions_A.csv
python run_agent_tools.py --lang en --inputs benchmark_inputs_en.csv --model <model> --runs 3 --out predictions_B.csv
python evaluate.py --inputs benchmark_inputs_en.csv --meta benchmark_meta_en.csv --pred predictions_A.csv predictions_B.csv --confusion
```

다른 팀의 에이전트도 같은 형식(`case_id, run, action, reason`)의 예측 파일만 내면 같은 기준으로 채점된다. 트랙 B는 `tools` 열과 대화 전체 기록(`.jsonl`)이 함께 남아 오류 분석에 쓸 수 있다.

5. 강의(Evaluating LLMs) 내용과의 대응

강의 내용	이 벤치마크에서의 반영
Close-ended 과제는 자동 평가가 가능 (분류)	세 행동 중 하나를 고르는 분류 과제로 정의해 정답과 자동 비교. 근거(reason)는 오류 분석용으로만 사용
Loss는 실제 과제 성능을 반영하지 못함	과제 목적에 맞춘 지표 설계: 불필요 개입률, 탐지율, 오류별 비용
구현 방식(프롬프트·생성 방식)에 따라 결과가 달라짐	모든 모델에 같은 시스템 프롬프트, 같은 JSON 형식, 같은 temperature, 같은 도구 프로토콜 사용. 형식 위반은 무효로 처리
Contamination, 비공개 테스트셋 권장	정답과 메타 파일을 입력과 분리. Phase 3에서 다른 팀에게는 입력과 형식만 공개하고 정답은 비공개로 유지
Goodhart의 법칙, 숫자만 쫓는 문제	단일 지표가 아니라 여러 지표를 함께 성공 기준으로 사용 (3.3절). 비용 점수도 질문만 하는 찍기 전략에 속지 않도록 가중치를 점검하고 민감도 분석을 함께 보고 (3.2절)
Human evaluation: 라벨러 간 불일치	독립 라벨러 2명의 Cohen's kappa로 라벨 타당성 확인
에이전트 벤치마크: 답 하나가 아니라 궤적·도구·환경을 평가	트랙 B: 도구로 정보를 모으는 여러 턴의 상호작용, 도구 호출 예산(최대 4회), 궤적 기록과 도구 사용 지표
pass@k, pass^k, 반복 시행과 95% CI	사례당 3회 실행해 일관성 측정, 정확도에 부트스트랩 95% CI 표시
Reward hacking, 약한 검증기	지름길 분석으로 "맥락 없이 맞히는 길"을 찾아 막음. 빈 답·형식 오류는 성공으로 세지 않음
점수는 모델 + 하네스 + 예산의 조합	실행기(하네스)를 하나로 고정하고 모델만 바꿔 비교. 보고 시 모델·온도·실행 횟수·도구 한도 명시

6. 현재 상태와 한계

아직 하지 않은 것

- 실제 LLM 에이전트 결과: 입력(관찰 기록, 사용자 상태)이 바뀌어 이전 EXAONE 결과는 쓸 수 없고, 현재 버전 기준 결과(트랙 A·B, EXAONE·Claude)는 아직 없다. 위 표는 기준선만 포함한다.
- 라벨러 2명의 독립 라벨링과 kappa 계산

1. **라벨 타당성:** 정답은 설계자의 판단이다. kappa로 검증하기 전까지는 타당성을 주장할 수 없다.
2. **합성 신호:** 사용량 숫자 4개, 기분, 평소 사용량만 실제 데이터이고, 스와이프·체류·선언 목표·사용자 상태·관찰 기록·터치 시퀀스는 합성이다. 실제 사용자의 개입 선호를 수집한 데이터가 아니며, 감지 층이 완벽하다고 가정한다(1.4절).
3. **표본 크기:** 81개라 정확도 신뢰구간 폭이 약 ±0.1이다. 0.1보다 작은 차이는 해석하지 않는다.
4. **비용 가중치:** 오류 비용은 설계 가정이다. 초기 가중치의 허점(항상 질문하는 규칙이 1위)은 수정했지만, 놓침을 더 무겁게 보는 세트에서는 이 규칙이 다시 3위까지 올라간다. 그래서 cost_score는 단독으로 쓰지 않고 question_burden, ask_recall, 민감도 분석 결과와 함께 보고한다.
5. **번역:** 영어 문장은 한국어 원문을 번역한 것이라 미묘한 뉘앙스 차이가 있을 수 있다.
6. **누출 점검 범위:** 키워드 점검은 문체 같은 암묵적 단서를 잡지 못한다. 단일 단서 규칙과 B4 단어 분류기로 보완했지만, 여러 단서를 조합한 지름길까지 모두 배제하지는 못한다.